

ABSTRACT

This thesis investigates PINN-inspired neural surrogates for six-degree-of-freedom (6-DOF) launch vehicle ascent dynamics, with the goal of approximating trajectories generated by a high-fidelity solver, at substantially reduced computational cost. Starting from a rigid-body 6-DOF dynamics model and a direct optimal control formulation serving as a reference trajectory generator, a dataset of numerically optimal ascent trajectories is generated using collocation-based methods. These reference trajectories serve as supervision for training structured physics-informed regularization neural surrogates that predict full state evolution from time and mission context parameters.

The primary contribution of this work is the design and evaluation of a structured physics-based regularization neural surrogate architecture, referred to in this thesis as Direction AN, which combines Fourier time embeddings, a context encoder for physical parameters, a shared residual backbone, and specialized output branches for translational motion, rotational dynamics, and mass depletion. Physical structure is encouraged through soft physics-based regularization derived from a reduced, uncontrolled form of the equations of motion, rather than strict enforcement of full controlled dynamics, together with additional structural constraints such as quaternion normalization, mass monotonicity, and boundary consistency. A phased training strategy is employed to balance data fidelity, physics regularization, and smoothing terms, improving stability during learning.

The resulting surrogate accurately reconstructs the dominant qualitative and quantitative structure of the state trajectories over the launch phase (0–30 s) across a range of sounding-rocket configurations, while maintaining approximate physical plausibility under soft constraints relative to a high-fidelity truth integrator. Although the proposed model is not employed to guide or optimize control inputs in the present study, its differentiable structure and physics-regularized formulation

conceptually suggest potential suitability for future integration into gradient-based trajectory optimization frameworks, provided that the architecture is extended to explicitly condition on control inputs and to enforce the full controlled dynamics. Results indicate that constructing a neural surrogate with precise characteristics of physical phenomena allows for the recreation of rocket ascent data in a way comparable to actual data collected via rocket launches. This will provide the capability to analyse launch trajectories quickly and affordably while providing opportunities for improvement of said trajectories.

CHAPTER I

INTRODUCTION

1.1 Background and Motivation

Trajectory optimization for rocket ascent is a foundational problem in aerospace engineering, involving the determination of thrust and steering profiles that guide a launch vehicle from liftoff to a desired terminal state while satisfying nonlinear dynamics, operational constraints, and structural limits. Classical solution approaches are well established and include indirect methods derived from Pontryagin’s Maximum Principle as well as direct transcription techniques such as collocation and pseudospectral methods, which convert the continuous-time optimal control problem into a large-scale nonlinear programming problem [3, 8, 2].

While these methods are theoretically rigorous and widely used, they rely on repeated numerical integration of high-dimensional nonlinear ordinary differential equations and the solution of constrained optimization problems with thousands of decision variables. As a result, they can become computationally expensive in repeated-evaluation settings, sensitive to initialization, and less amenable to direct embedding in higher-level loops when repeated evaluation or gradient access is required within higher-level design loops, uncertainty analysis, or real-time decision-making frameworks. These limitations become particularly pronounced when repeated trajectory evaluations or gradient information are required, such as in design space exploration or optimization under uncertainty.

To address these challenges, various surrogate modeling strategies have been investigated as practical alternatives for approximating rocket dynamics while lowering computational demands. There are many different common methods used to increase the speed of simulations, including; (1) Response Surface Methods, (2)

Reduced Order Models, and (3) Fully Data-driven Neural Networks. Many times these methods provide enormous improvements in simulation speed compared to traditional approaches; however, these same methods have also been shown in a number of cases to lack robustness and therefore do not provide sufficient assurance that what is predicted outside the training conditions will be physically reasonable.

Meanwhile other techniques under development within the field of Scientific Machine Learning have led to the emergence of "Physics-Informed Neural Networks", and more broadly a number of other types of Physics Regularized Learning Models, all of which use physical knowledge during the training of a model. Instead of simply fitting a model to observed data, these methods utilize additional constraints to allow the model to take on additional information from the governing physics equations governing system behaviour. This formulation allows Neural Networks to maintain their flexibility while being guided by both observed data and the underlying physical laws that govern all physical systems. The rapidly growing intersection between Surrogate Modelling techniques and Physics-Informed learning techniques, therefore, provides an ideal impetus to re-evaluate Rocket Ascent Modelling through the use of a Machine Learning lens.

1.1.1 State of the Art

In this thesis, the term PINN-inspired is used to denote physics-regularized learning architectures that incorporate physical structure through soft constraints, rather than strict equation enforcement.

Physics-informed neural networks were originally proposed as a means of solving forward and inverse problems governed by differential equations by embedding the governing equations directly into the learning objective. Since their introduction, PINNs have been applied to a wide range of problems in fluid dynamics, solid mechanics, and control, and extensive surveys have documented both their potential and their limitations. In particular, prior work has highlighted challenges related

to optimization stiffness, loss balancing, long-horizon accuracy, and constraint enforcement [7, 11].

Analytical and empirical studies have further shown that PINN performance is highly sensitive to architectural choices, training strategies, and the relative weighting of data and physics losses. While PINNs can approximate smooth solution manifolds effectively, they do not generally provide hard guarantees of constraint satisfaction when physical laws are enforced through penalty-based formulations [11]. These limitations are particularly pronounced when the governing equations are nonlinear and coupled, as is the case for rocket ascent dynamics.

In aerospace applications, learning-based surrogate models have traditionally been used for aerodynamic prediction, reduced-order dynamics, or narrow operating regimes. The application of physics-informed learning to trajectory optimization problems remains comparatively limited, particularly for six-degree-of-freedom (6-DOF) rocket ascent dynamics, which involve strong nonlinearities, coupling between translational and rotational motion, and mass depletion effects.

Existing studies in this area have primarily focused on forward or inverse dynamics problems, while fewer works investigate structured neural surrogates trained on optimal trajectories as approximations to high-fidelity numerical integration. This gap is especially relevant for ascent problems, where repeated trajectory evaluation is computationally demanding and where physical structure is well understood but difficult to enforce strictly within learning-based models [7, 11].

1.1.2 Rationale

The central rationale of this thesis is to investigate whether a structured, PINN-inspired, physics-regularized neural network can serve as an effective surrogate for rocket ascent trajectory reconstruction. Rather than learning a closed-form state-transition function or directly optimizing control inputs, the proposed approach aims to approximate optimal ascent trajectories as explicit functions of time and mission

parameters.

Surrogates that ascertain the approximate behavior of a trajectory can be generated quickly and differentially (allowing gradients to be computed) by utilizing the computational load during the offline training stage rather than performing the numerical integration task at each iteration during the inference stage. In addition, to provide further opportunities for physical interpretability, physics-based regularization is performed by incorporating 'soft' residual penalties (in fact, the residuals are derived from a reduced form of the governing equations) to ensure that the approximate behavior produced by the surrogate is both physical plausible and retains the flexibility to be modified through the data-driven learning process.

This formulation is particularly suited to ascent analysis and preliminary design studies, where fast evaluation and smooth sensitivity information are valuable, but strict enforcement of all physical constraints may be relaxed. The surrogate is not intended to replace the optimal control solver, but to approximate its output trajectories under similar conditions. The study therefore focuses on assessing the accuracy, stability, and physical plausibility of trajectory reconstruction under soft constraints, rather than on direct deployment in closed-loop control or safety-critical applications.

1.1.3 Problem Statement

The problem addressed in this thesis is the development of a PINN-inspired, physics-regularized trajectory surrogate capable of approximating three-dimensional rocket ascent behavior during the launch phase. The surrogate is trained on full-state trajectories generated from high-fidelity numerical simulations of a 6-DOF rigid-body dynamics model and is designed to reproduce the dominant coupled translational, rotational, and mass-depletion characteristics of ascent under soft physics-based regularization.

While the broader motivation is to support future trajectory optimization work-

flows, the scope of this study is limited to trajectory reconstruction and physical consistency analysis, rather than the direct computation of optimal control inputs using the learned surrogate.

1.1.4 Objectives

The specific objectives of this thesis are as follows:

1. To formulate a three-dimensional rocket ascent model suitable for physics-regularized learning.
2. To design a structured PINN-inspired architecture incorporating Fourier time embeddings, context encoding, and residual multilayer perceptrons.
3. To train the surrogate using full-state trajectory data generated from high-fidelity numerical simulations relative to the surrogate model.
4. To evaluate the accuracy, stability, and approximate physical consistency of the learned surrogate during the launch phase under soft physics-based constraints.
5. To assess, at a conceptual level, the suitability of the surrogate as a building block for potential future integration into constrained trajectory optimization frameworks, once extended to explicitly condition on control inputs and enforce the full controlled dynamics.

1.1.5 Scope and Limitations

The scope of this work is limited to three-dimensional rocket ascent dynamics without wind disturbances. The surrogate is trained on trajectory data covering the initial 30 seconds of flight, corresponding to the launch phase. As a result, conclusions regarding full-ascent modelling, closed-loop control optimization, and generalization beyond the training regime are outside the scope of this thesis.

In particular, the study does not demonstrate closed-loop optimization of control inputs using the learned surrogate. These limitations are acknowledged and discussed as directions for future work.

1.1.6 Research Framework

The research framework follows a multi-level structure. At the data generation level, high-fidelity numerical simulations and optimal control solvers are used to produce dynamically feasible ascent trajectories. At the learning level, these trajectories are used to train a PINN-inspired, physics-regularized neural surrogate that approximates state evolution over time. At the application level, the trained surrogate is evaluated as a candidate approximation to numerical integration in future optimization and guidance workflows, rather than as a full replacement for a high-fidelity dynamics model.

1.1.7 Structure of the Thesis

Chapter 2 presents the theoretical background, including physics-informed neural networks, optimal control formulations, and the rocket dynamics model, followed by a detailed description of the proposed architecture and training methodology. Chapter 3 reports the numerical results. Chapter 4 discusses the implications and limitations of the findings. Chapter 5 concludes the thesis and outlines directions for future work.

CHAPTER II

MATERIALS AND METHODS

2.1 Overview of the Methodological Workflow

The methodology was organized around three sequential stages. First, a physical model of six-degree-of-freedom (6-DOF) rocket dynamics was formulated and implemented numerically. Second, optimal control trajectories were generated using direct collocation methods and used to construct training datasets. Third, PINN-inspired surrogates were trained on these datasets to approximate optimal ascent trajectories implied by the dynamics. The relationship between the optimal control solver, truth integrator, and PINN surrogate is illustrated schematically in Figure 2.1.

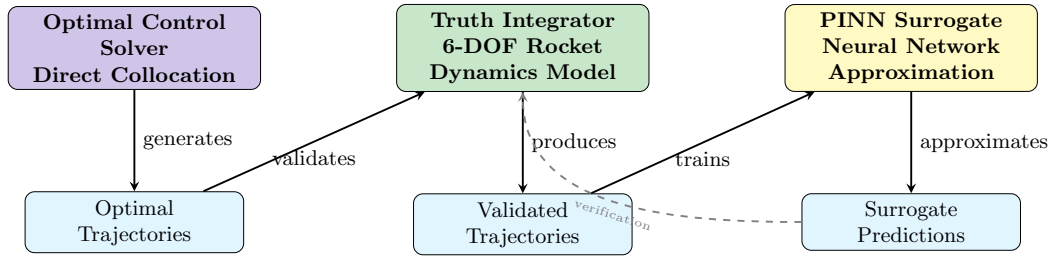


Figure 2.1: Schematic diagram illustrating the relationship between the optimal control solver, truth integrator, and PINN surrogate in the methodological workflow.

2.2 Rocket Dynamics Model

2.2.1 Reference Frames and Coordinate Systems

Two primary reference frames were used. The inertial frame was defined as Earth-centred and non-rotating, with the z -axis pointing upward (opposite to gravity). The body frame was defined as vehicle-fixed, with the x -axis aligned with the nomi-

nal thrust direction. Transformations between frames were represented using unit quaternions.

The state vector $\mathbf{x}(t) \in \mathbb{R}^{14}$ was defined as

$$\mathbf{x}(t) = \begin{bmatrix} \mathbf{r}_i^\top & \mathbf{v}_i^\top & \mathbf{q}^\top & \boldsymbol{\omega}_b^\top & m \end{bmatrix}^\top,$$

where $\mathbf{r}_i \in \mathbb{R}^3$ was the position in the inertial frame, $\mathbf{v}_i \in \mathbb{R}^3$ was the inertial velocity, $\mathbf{q} = [q_0, q_1, q_2, q_3]^\top$ was a unit quaternion representing the attitude (body-to-inertial rotation), $\boldsymbol{\omega}_b \in \mathbb{R}^3$ was the angular velocity expressed in the body frame, and $m \in \mathbb{R}$ was the mass of the vehicle.

The control vector $\mathbf{u}(t) \in \mathbb{R}^4$ was defined as

$$\mathbf{u}(t) = \begin{bmatrix} T & \theta_g & \phi_g & \delta \end{bmatrix}^\top,$$

where T was the commanded thrust magnitude, θ_g and ϕ_g were the thrust gimbal pitch and yaw angles, and δ was a representative control surface deflection.

The body-to-inertial rotation matrix $R_{b \rightarrow i}(\mathbf{q})$ was obtained from the quaternion $\mathbf{q}$. Its transpose $R_{i \rightarrow b} = R_{b \rightarrow i}^\top$ transformed vectors from the inertial frame to the body frame.

2.2.2 Equations of Motion

The translational motion was governed by Newton's second law:

$$\dot{\mathbf{r}}_i = \mathbf{v}_i, \quad \dot{\mathbf{v}}_i = \frac{1}{m} \mathbf{F}_i(\cdot) + \mathbf{g}_i(\mathbf{r}_i),$$

where $\mathbf{F}_i$ was the total non-gravitational force in the inertial frame and $\mathbf{g}_i$ was the gravitational acceleration. A constant gravitational field $\mathbf{g}_i = [0, 0, -g_0]^\top$ was employed, where $g_0 = 9.81 \text{ m/s}^2$ was the nominal surface gravity.

Rotational dynamics were expressed as

$$\dot{\mathbf{q}} = \frac{1}{2} \mathbf{q} \otimes \begin{bmatrix} 0 \\ \boldsymbol{\omega}_b \end{bmatrix}, \quad \dot{\boldsymbol{\omega}}_b = \mathbf{I}_b^{-1} \left(\mathbf{M}_b - \boldsymbol{\omega}_b \times (\mathbf{I}_b \boldsymbol{\omega}_b) \right),$$

where $\mathbf{I}_b$ was the inertia tensor in the body frame and $\mathbf{M}_b$ was the sum of aerodynamic and thrust moments.

The mass dynamics followed the standard rocket equation [9]:

$$\dot{m} = -\frac{T}{I_{sp} g_0},$$

with specific impulse I_{sp} and surface gravity g_0 . A dry-mass limit was enforced in the implementation to prevent unphysical mass depletion.

These equations follow standard formulations for rigid-body rocket dynamics [3, 2].

2.2.3 Aerodynamic and Propulsion Models

The atmosphere was modelled by an exponential density profile:

$$\rho(h) = \rho_0 \exp\left(-\frac{h}{h_{scale}}\right),$$

where h was altitude above sea level, $\rho_0 = 1.225 \text{ kg/m}^3$ was the sea-level density, and $h_{scale} = 8400 \text{ m}$ was a scale height. The dynamic pressure was computed as

$$q_{dyn} = \frac{1}{2} \rho(h) \|\mathbf{v}_{rel,b}\|^2,$$

where $\mathbf{v}_{rel,b}$ was the relative wind velocity in the body frame.

Aerodynamic forces were naturally expressed in the body frame. The relative

wind velocity was computed as

$$\mathbf{v}_{\text{rel},i} = \mathbf{v}_i - \mathbf{v}_{\text{wind},i}, \quad \mathbf{v}_{\text{rel},b} = R_{i \rightarrow b}(\mathbf{q}) \mathbf{v}_{\text{rel},i},$$

where $\mathbf{v}_{\text{wind},i}$ was the wind velocity in the inertial frame. Wind disturbances were neglected, so $\mathbf{v}_{\text{wind},i} = \mathbf{0}$.

Drag and lift forces in the body frame took the form

$$\mathbf{F}_{D,b} = -q_{\text{dyn}} S_{\text{ref}} C_D \hat{\mathbf{v}}_{\text{rel},b}, \quad \mathbf{F}_{L,b} = q_{\text{dyn}} S_{\text{ref}} C_{L\alpha} \alpha \hat{\mathbf{e}}_L,$$

where S_{ref} was a reference area, C_D was the drag coefficient, $C_{L\alpha}$ was a lift-curve slope, α was the angle of attack computed from $\mathbf{v}_{\text{rel},b}$, and $\hat{\mathbf{e}}_L$ was a unit lift direction approximately perpendicular to the body x -axis and the relative velocity.

The thrust vector in the body frame was

$$\mathbf{u}_T = \begin{bmatrix} \cos \theta_g \cos \phi_g \\ \sin \phi_g \\ \sin \theta_g \cos \phi_g \end{bmatrix}, \quad \mathbf{F}_{T,b} = T \frac{\mathbf{u}_T}{\|\mathbf{u}_T\|},$$

so that the total non-gravitational force in the body frame was

$$\mathbf{F}_b = \mathbf{F}_{T,b} + \mathbf{F}_{D,b} + \mathbf{F}_{L,b}, \quad \mathbf{F}_i = R_{b \rightarrow i}(\mathbf{q}) \mathbf{F}_b.$$

2.3 Numerical Trajectory Generation via Optimal Control

2.3.1 Optimal Control Problem Formulation

Numerically optimal ascent trajectories were generated by solving a constrained optimal control problem (OCP). The OCP was discretised using a direct collocation method based on the Hermite–Simpson scheme [2, 8]. Which served as a data-generation mechanism for training and evaluating the physics-informed surrogate

model. The OCP was formulated as a continuous-time optimal control problem over a finite horizon $t \in [0, t_f]$, with the 6-DOF dynamics as constraints. The objective function was formulated to minimise fuel consumption, subject to path and terminal constraints on altitude, velocity, attitude, and structural loads. The time horizon was fixed at $t_f = 30$ s.

The control vector $\mathbf{u}(t) \in \mathbb{R}^4$ was defined as

$$\mathbf{u}(t) = \begin{bmatrix} T & \theta_g & \phi_g & \delta \end{bmatrix}^\top,$$

with bounds on thrust magnitude $T \in [0, T_{\max}]$, gimbal angles $\theta_g, \phi_g \in [-\theta_{\max}, \theta_{\max}]$, and control surface deflection $\delta \in [-\delta_{\max}, \delta_{\max}]$. Path constraints enforced maximum dynamic pressure $q_{\text{dyn}} \leq q_{\text{dyn},\max}$ and maximum load factor $n \leq n_{\max}$.

The OCP was discretised using a direct collocation method. The continuous state and control trajectories were approximated on a uniform grid of nodes $\{t_k\}_{k=0}^N$ with step size $h = t_f/N$. At each node, the state $x_k \approx x(t_k)$ and control $u_k \approx u(t_k)$ became decision variables in a finite-dimensional nonlinear programme (NLP). The dynamics were enforced via Hermite–Simpson collocation, which provided third-order accuracy by introducing a collocation point at the midpoint of each interval and enforcing a defect constraint of the form

$$x_{k+1} = x_k + \frac{h}{6} (f(x_k, u_k) + 4f(x_m, u_m) + f(x_{k+1}, u_{k+1})),$$

where $f(x, u)$ denotes the state derivative, x_m is a midpoint state constructed from (x_k, x_{k+1}) and $(f(x_k, u_k), f(x_{k+1}, u_{k+1}))$, and u_m is the midpoint control. The resulting collocation defects were enforced as equality constraints in the NLP.

2.3.2 Numerical Solvers and Software

The 6-DOF dynamics, including aerodynamics, thrust, and mass depletion, were implemented symbolically in CasADi. This provided exact Jacobians and Hessians

to the NLP solver, which was crucial for robustness and performance in the presence of stiff dynamics and tight path constraints. Special care was taken in the implementation to avoid non-differentiabilities and division by small quantities, for example by using smooth norm approximations and explicit clamping of mass and thrust.

CasADi version 3.6.7 and IPOPT version 3.14.16 were used in all numerical optimization experiments, with linear solver backends selected automatically based on availability.

The discretised OCP was solved using IPOPT, a large-scale interior-point non-linear optimiser. State and control variables were scaled using reference length, velocity, time, mass, and force scales for conditioning, and linear solver backends (such as MUMPS or HSL variants) were selected automatically depending on availability. The framework supported both fixed and free final time, with appropriate bounds on t_f when treated as a decision variable.

Path constraints on dynamic pressure q_{dyn} , load factor n , and mass were enforced directly in the NLP using auxiliary CasADi functions for these quantities. Operational limits on gimbal angles, control-surface deflections, and angle of attack were also encoded in the state and control bounds.

In addition to the collocation-based OCP solver, a high-fidelity C++ integrator for the same 6-DOF dynamics was implemented and used as a truth model for validation and data generation. This integrator supported inverse-square gravity, exponential atmosphere, aerodynamic forces and moments, wind callbacks, and detailed diagnostic outputs such as dynamic pressure, load factor, and constraint violations.

For data-driven approximation, a differentiable version of the dynamics and the physics-informed neural network models were implemented using PyTorch [6] version 2.x (or LibTorch 2.3.1 as an C++ frontend for PyTorch). This module mirrored the CasADi model, including the state and control definitions, aerodynamic coefficients, and mass dynamics, but used tensor operations to enable formal automatic

differentiation of the surrogate.

The time discretization was fixed at $N = 1501$ points, corresponding to a 30-second flight duration sampled at 50 Hz.

2.4 Dataset Construction and Preprocessing

2.4.1 Dataset Structure

The data pipeline produced a hierarchy of datasets. At the lowest level, raw cases were stored as individual HDF5 files containing time grids, full 14-D state trajectories, control histories, monitor signals (e.g., dynamic pressure and load factor), and rich metadata including physical parameters, solver statistics, and configuration hashes. These raw cases were then aggregated into processed split files (train/validation/test) in nondimensional form.

The processed datasets contained approximately 120 training cases, 20 validation cases, and 20 test cases. Each trajectory was discretised on a uniform time grid with $N = 1501$ points, corresponding to a 30-second flight duration sampled at 50 Hz. Each processed dataset exposed a time grid, a context vector $\mathbf{c} \in \mathbb{R}^7$ encoding key physical and environmental parameters (initial mass m_0 , specific impulse I_{sp} , drag coefficient C_D , lift-curve slope $C_{L\alpha}$, pitch-moment coefficient $C_{m\alpha}$, maximum thrust T_{max} , and normalised state targets. Although the context vector includes a wind magnitude parameter for extensibility, wind disturbances were set to zero in all experiments reported in this thesis.

The state tensor shape was $\text{state} \in \mathbb{R}^{N_{\text{cases}} \times N_{\text{time}} \times 14}$, where N_{cases} is the number of trajectories, $N_{\text{time}} = 1501$ is the number of time points, and 14 is the state dimension.

2.4.2 Nondimensionalization and Scaling

All state and control variables were nondimensionalized using physics-aware reference scales for numerical conditioning and intended to be scale-invariant learning. The reference scales were chosen to yield quantities of order unity, as summarized

in Table 2.1.

Table 2.1: Reference scales for nondimensionalization

Quantity	Symbol	Value	Motivation
Length	L_{ref}	10,000 m	Typical altitude
Velocity	V_{ref}	313 m/s	$\sqrt{g_0 L_{\text{ref}}}$
Time	T_{ref}	31.62 s	$L_{\text{ref}}/V_{\text{ref}}$
Mass	M_{ref}	50 kg	Nominal wet mass
Force	F_{ref}	490 N	$M_{\text{ref}}g_0$

The scaling equation was

$$\tilde{x} = \frac{x}{x_{\text{ref}}},$$

where $\tilde{x}$ is the nondimensionalized quantity and x_{ref} is the corresponding reference scale. This nondimensionalization ensured that all state components were typically in the range $[0.1, 10]$, which improved gradient flow during neural network training and reduced numerical errors in the optimization solver. The scaling was inverted during evaluation to recover dimensional quantities. This nondimensionalization follows standard practice in numerical simulation for conditioning and training stability [1].

2.5 Physics-Informed Neural Network Model

Unlike classical PINNs that enforce differential equations via automatic differentiation at collocation points, the present approach employs finite-difference approximations of temporal derivatives on discretized trajectories, and is therefore described as PINN-inspired rather than a strict PINN formulation.

2.5.1 Model Inputs and Outputs

The model inputs were time t (a scalar) and a context vector $\mathbf{c} \in \mathbb{R}^7$ encoding physical and environmental parameters. The model output was the state vector $\mathbf{x}(t) \in \mathbb{R}^{14}$.

2.5.2 Network Architecture (Direction AN)

The Direction AN architecture was organized into three stages (see Figure 2.2). A shared stem first embedded the normalised time and context: time was expanded and encoded by Fourier features embedding [10] with 8 frequencies, while the context vector was passed through a small encoder (a shallow multilayer perceptron) and then concatenated with the time embedding. This combined input was processed by a residual multilayer perceptron with 4 layers, hidden dimension 128, skip connections, tanh activation, and layer normalization, yielding a latent feature sequence $z(t, c)$ of fixed dimension along the trajectory.

On top of this shared latent representation, three mission branches specialized to different parts of the 14-D state. A translation branch with hidden dimensions [128, 128] mapped z to position and velocity components $[\mathbf{r}_i, \mathbf{v}_i]$. A rotation branch with hidden dimensions [256, 256] mapped z to quaternion and angular velocity components $[\mathbf{q}, \boldsymbol{\omega}_b]$ with explicit quaternion renormalization. A mass branch with hidden dimension [64] produced the mass trajectory $m(t)$. The concatenation of these branch outputs formed the predicted state $x_\theta(t)$.

2.5.3 Physics Residual Computation

Although control inputs were explicitly modeled in the optimal control formulation used for data generation, the trained physics-informed neural network surrogate did not take control variables as direct inputs. Consequently, control terms were omitted from the physics residual during training, and the surrogate learned the state evolution implicitly from the resulting state trajectories. This design choice was adopted in order to evaluate the ability of the surrogate to reproduce ascent dynamics without explicit control conditioning.

The physics residual was computed using finite difference derivative approximations, rather than automatic differentiation. For interior points, a central differ-

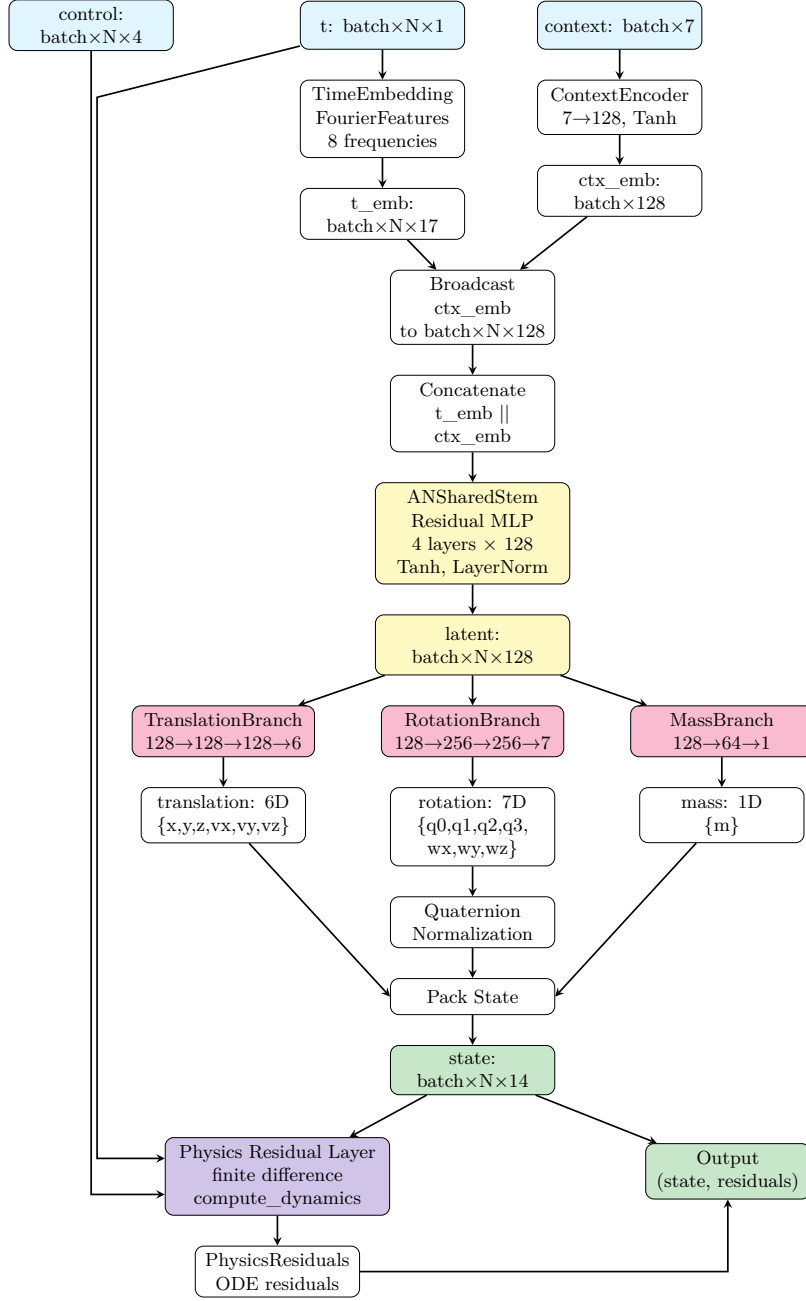


Figure 2.2: Architecture of the Direction AN model. The shared stem processes architecture of the Direction AN model. The shared stem processes time and context inputs to produce a latent representation $z(t, c)$. Three specialized branches (translation, rotation, mass) map this latent space to the corresponding state components. The physics residual layer computes dynamics residuals to enforce physical consistency.

ence scheme was used:

$$\frac{dx}{dt} \approx \frac{x_{i+1} - x_{i-1}}{2\Delta t},$$

where $\Delta t = 0.02$ s is the time step (corresponding to 50 Hz sampling). For boundary points, forward difference (first point) and backward difference (last point) schemes were used.

In the PINN setting, explicit control inputs were omitted and set to zero in the physics residual, reflecting the design choice that the network learned state evolution implicitly from trajectories rather than from control commands. The control $\mathbf{u}_k$ in the physics loss was therefore set to zero ($\mathbf{u}_k = [0, 0, 0, 0]^\top$). It is emphasized that this residual does not correspond to the true flight dynamics; instead, it acts as a structural regularizer that biases the learned trajectories toward smooth, physically plausible manifolds consistent with the reduced, uncontrolled dynamics. It is important to note that this zero-control assumption was consistent with the processed dataset format, which did not include control trajectories.

As a result, the physics residual does not represent the full controlled dynamics, but instead acts as a regularizer that biases the learned trajectories toward consistency with the uncontrolled structure of the governing equations. Because control inputs are omitted, the residual does not represent the true flight dynamics, but instead biases the learned trajectories toward a physically structured manifold.

Accordingly, the residual should be interpreted as a physics-based regularizer rather than an enforcement of the full controlled equations of motion.

2.5.4 Architecture Selection Procedure

The final Direction AN architecture was obtained through a structured model selection process. Several candidate architectures were evaluated to assess the impact of architectural choices on training stability, physical consistency, and trajectory reconstruction accuracy.

In particular, variants differing in the handling of temporal inputs, mission parameters, and output coupling were examined. Architectures combining time and context inputs at a single shallow layer were evaluated and not retained due to unstable convergence behavior and sensitivity to scaling. In contrast, architectures employing separate encoders for temporal inputs and mission parameters, followed by a shared latent representation, showed improved convergence behavior and more stable physics residuals. These architectures were retained for further evaluation.

Additionally, separating the output into distinct heads for translational states, rotational states, and mass evolution was evaluated and found to facilitate loss balancing and improve interpretability of individual error components. Based on these observations, a shared-stem architecture with specialized output branches was selected as the final Direction AN configuration, as illustrated in Figure 2.3.

2.6 Training Procedure

2.6.1 Loss Function Definition

The total training loss was constructed as a weighted combination of multiple terms enforcing data fidelity, physics-based regularization, and structural constraints. Rather than relying on a single objective, the loss function was designed to reflect different aspects of the rocket ascent dynamics following standard physics-informed learning formulations [7].

In addition to the core data and physics losses, several auxiliary regularization terms were introduced as inductive biases to stabilize training and encode known symmetries of the ascent problem; these terms are not intended to represent independent physical laws. The primary loss components were grouped into four categories:

1. Data fidelity losses, which penalized deviations between predicted and reference state trajectories obtained from numerical simulation.

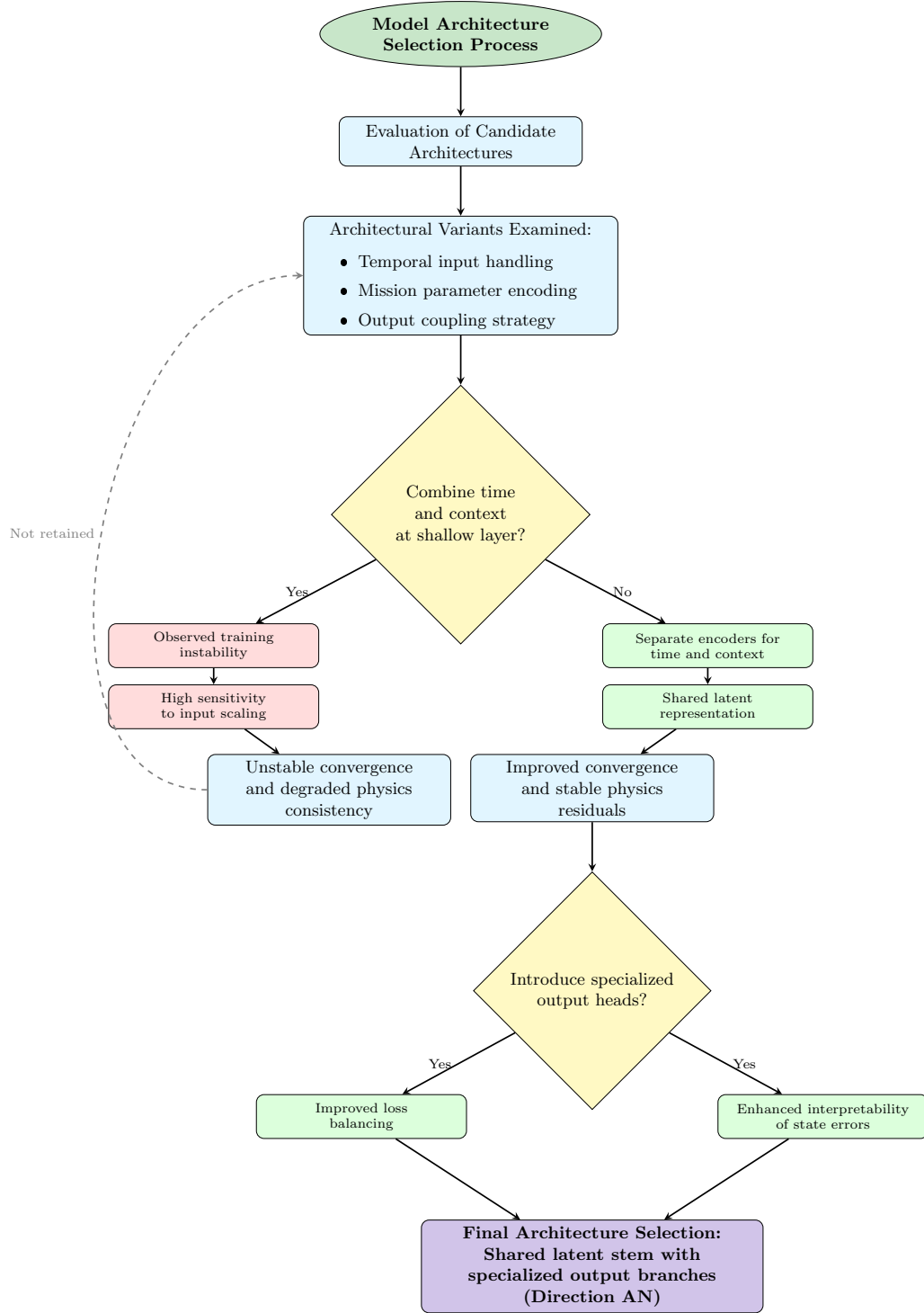


Figure 2.3: Model selection process for the Direction AN architecture. Summarizes the selection process used to refine the network architecture. Architectural choices were guided by observed training stability, sensitivity to input scaling, and the ability to balance heterogeneous loss components, rather than by exhaustive hyperparameter search.

2. Physics-based regularization losses, which penalized violations of a reduced form of the governing equations of motion, including translational dynamics, rotational kinematics, and mass depletion.
3. Structural constraint losses, which enforced properties intrinsic to the system representation, such as unit-norm quaternion constraints and monotonic mass depletion.
4. Regularization losses, which promoted smoothness of the predicted trajectories and suppressed high-frequency numerical artifacts.

The data loss L_{data} was a component-weighted mean-squared error between predicted and true states, with separate group weights for translation, rotation, and mass:

$$L_{\text{data}} = \lambda_{\text{translation}} L_{\text{translation}} + \lambda_{\text{rotation}} L_{\text{rotation}} + \lambda_{\text{mass}} L_{\text{mass}},$$

where $\lambda_{\text{translation}} = 1.0$, $\lambda_{\text{rotation}} = 1.0$, and $\lambda_{\text{mass}} = 1.0$ were the group weights, and $L_{\text{translation}}$, L_{rotation} , and L_{mass} were component-weighted MSE losses for the respective state groups.

The physics loss L_{phys} penalised violations of the continuous dynamics by comparing a finite-difference estimate of the time derivative $\dot{x}_{\theta}(t_k)$ to the right-hand side $f(x_{\theta}(t_k), \mathbf{u}_k = \mathbf{0}; \mathbf{p})$ of the 6-DOF model implemented in PyTorch:

$$L_{\text{phys}} = \frac{1}{N} \sum_{k=0}^{N-1} \|\dot{x}_{\theta}(t_k) - f(x_{\theta}(t_k), \mathbf{u}_k = \mathbf{0}; \mathbf{p})\|^2.$$

A boundary term L_{bc} enforced agreement between predicted and true initial conditions:

$$L_{\text{bc}} = \|\mathbf{x}_{\theta}(0) - \mathbf{x}^*(0)\|^2.$$

Additional loss terms were included to suppress non-physical lateral drift, enforce smooth ascent behavior, and stabilize training by penalizing high-frequency

numerical artifacts, particularly during the early ascent phase. These loss terms are:

- $L_{\text{mass,res}}$: Mass residual loss ensuring mass decreases monotonically
- $L_{\text{vz,res}}$: Vertical velocity residual loss
- $L_{\text{vxy,res}}$: Horizontal velocity residual loss
- $L_{\text{smooth,z}}$: Temporal smoothing for altitude
- $L_{\text{smooth,vz}}$: Temporal smoothing for vertical velocity
- $L_{\text{pos,vel}}$: Position–velocity consistency loss
- $L_{\text{smooth,pos}}$: Position smoothing loss
- $L_{\text{zero,vxy}}$: Zero horizontal velocity loss
- $L_{\text{zero,axy}}$: Zero horizontal acceleration loss
- L_{hacc} : Horizontal acceleration constraint loss
- $L_{\text{xy,zero}}$: Zero horizontal position loss

The total loss was

$$\begin{aligned}
\mathcal{L} = & \lambda_{\text{data}} L_{\text{data}} + \lambda_{\text{phys}} L_{\text{phys}} + \lambda_{\text{bc}} L_{\text{bc}} \\
& + \lambda_{\text{mass,res}} L_{\text{mass,res}} + \lambda_{\text{vz,res}} L_{\text{vz,res}} + \lambda_{\text{vxy,res}} L_{\text{vxy,res}} \\
& + \lambda_{\text{smooth,z}} L_{\text{smooth,z}} + \lambda_{\text{smooth,vz}} L_{\text{smooth,vz}} + \lambda_{\text{pos,vel}} L_{\text{pos,vel}} \\
& + \lambda_{\text{smooth,pos}} L_{\text{smooth,pos}} + \lambda_{\text{zero,vxy}} L_{\text{zero,vxy}} + \lambda_{\text{zero,axy}} L_{\text{zero,axy}} \\
& + \lambda_{\text{hacc}} L_{\text{hacc}} + \lambda_{\text{xy,zero}} L_{\text{xy,zero}},
\end{aligned} \tag{2.1}$$

where the loss weights were set to $\lambda_{\text{data}} = 1.0$, $\lambda_{\text{phys}} = 0.1$, $\lambda_{\text{bc}} = 1.0$, $\lambda_{\text{mass,res}} = 0.05$, $\lambda_{\text{vz,res}} = 0.05$, $\lambda_{\text{vxy,res}} = 0.01$, $\lambda_{\text{smooth,z}} = 5.0 \times 10^{-4}$, $\lambda_{\text{smooth,vz}} = 1.0 \times 10^{-4}$,

$\lambda_{\text{pos,vel}} = 1.0$, $\lambda_{\text{smooth,pos}} = 2.0 \times 10^{-3}$, $\lambda_{\text{zero,vxy}} = 1.0$, $\lambda_{\text{zero,axy}} = 1.0$, $\lambda_{\text{hacc}} = 0.02$, and $\lambda_{\text{xy,zero}} = 5.0$.

These auxiliary loss terms should be interpreted as inductive biases encoding known symmetries and qualitative properties of the ascent problem, rather than as substitutes for the governing equations themselves. Several constraints intentionally overlap in their effect to stabilize learning in symmetry-dominated dimensions and do not represent independent physical laws.

2.6.2 Optimization and Training Schedule

The network was trained using the Adam optimizer [4] with learning rate 1.0×10^{-3} , weight decay 1.0×10^{-5} , and batch size 8. Training was conducted for 160 epochs. A cosine annealing learning rate scheduler [5] was used with $T_{\text{max}} = 160$ and minimum learning rate $\eta_{\text{min}} = 1.0 \times 10^{-6}$.

A phased training schedule was employed where loss weights were adjusted over time. In the initial phase (approximately the first 55% of training epochs), the model focused on fitting the data and coarse physics constraints, with lower weights on higher-order regularization terms. In the second phase, these regularization weights were gradually increased using a cosine ramp schedule [5], which was designed to gradually increase the regularization weights over time, designing like this to allow the model to refine its predictions while maintaining physical consistency. This phased training schedule was designed to help the model refine its predictions while maintaining physical consistency.

Early stopping was implemented with patience 25 for phase 1 and patience 40 for phase 2, with minimum delta 0.0. This early stopping was designed to prevent the model from overfitting to the regularization terms early in training while ensuring that physical constraints are properly enforced in the final model.

2.6.3 Evaluation Metrics

Model performance was evaluated using root mean square error (RMSE) computed on the test set. For a trained Direction AN model, predictions $\hat{\mathbf{x}}_\theta(t_k)$ were generated for all test trajectories on a uniform time grid $\{t_k\}_{k=0}^{N-1}$ spanning the trajectory duration (30 seconds). The total RMSE was computed as

$$\text{RMSE}_{\text{total}} = \sqrt{\frac{1}{B \cdot N \cdot D} \sum_{b=1}^B \sum_{k=0}^{N-1} \sum_{d=1}^D \left(\hat{x}_\theta^{(b)}(t_k)_d - x^{*(b)}(t_k)_d \right)^2},$$

where $B = 20$ was the number of test trajectories, $N = 1501$ was the number of time points in the discretized trajectory grid (corresponding to a sampling rate of approximately 50 Hz), $D = 14$ was the state dimension, and $x^{*(b)}(t_k)$ denoted the ground truth state for trajectory b at time t_k . The summation over k from 0 to $N - 1$ averaged the squared errors across all time points in each trajectory, while the summation over b averaged across all test trajectories, and the summation over d averaged across all state components.

Per-component RMSE was computed by averaging over trajectories and time points for each state dimension d , while aggregated metrics were computed for translation (components 1–6: position and velocity), rotation (components 7–13: quaternion and angular velocity), and mass (component 14). All RMSE values were reported in nondimensional units, reflecting the scaled state representation used during training, and RMSE does not capture localized constraint violations, which are examined separately in Section 3.4

2.6.4 Loss Weight Selection and Sensitivity Analysis

The relative weighting of loss components was selected using a combination of physical reasoning and empirical sensitivity analysis. Initial loss weights were chosen based on the expected physical significance and numerical scale of each state

component, informed by exploratory analysis of the trajectory data. For example, altitude, vertical velocity, and mass evolution were assigned higher relative importance due to their dominant role in ascent performance.

To refine these initial choices, coarse parameter sweeps were conducted to identify ranges of loss weights that yielded stable training and physically plausible trajectories. Within these ranges, a lightweight evolutionary search procedure was employed to test local variations of loss weights and assess their effect on convergence behavior and validation error. This process was used to improve robustness rather than to identify a globally optimal configuration, as illustrated in Figure 2.4.

The final loss weights were selected based on a balance between data fidelity, physical consistency, and training stability, and were fixed for all reported experiments.

2.6.5 Implementation Details

The optimal control solver was implemented using CasADi for symbolic computation and IPOPT for nonlinear optimization. The truth integrator was implemented in C++. Random seeds were fixed for all experiments to ensure reproducibility. Data processing and training scripts were written in Python, using NumPy for numerical operations and HDF5 for data storage. The neural network models were implemented in PyTorch. Training was performed on a workstation with automatic device selection depending on availability. (CPU or GPU if available), and no hardware-specific optimizations were employed. This implementation was designed to be reproducible and scalable.

2.6.6 Summary of Chapter

A six-degree-of-freedom rocket dynamics model was formulated and implemented at three fidelity levels: a high-fidelity C++ truth integrator, a symbolic CasADi version for optimal control, and a differentiable PyTorch implementation for PINN

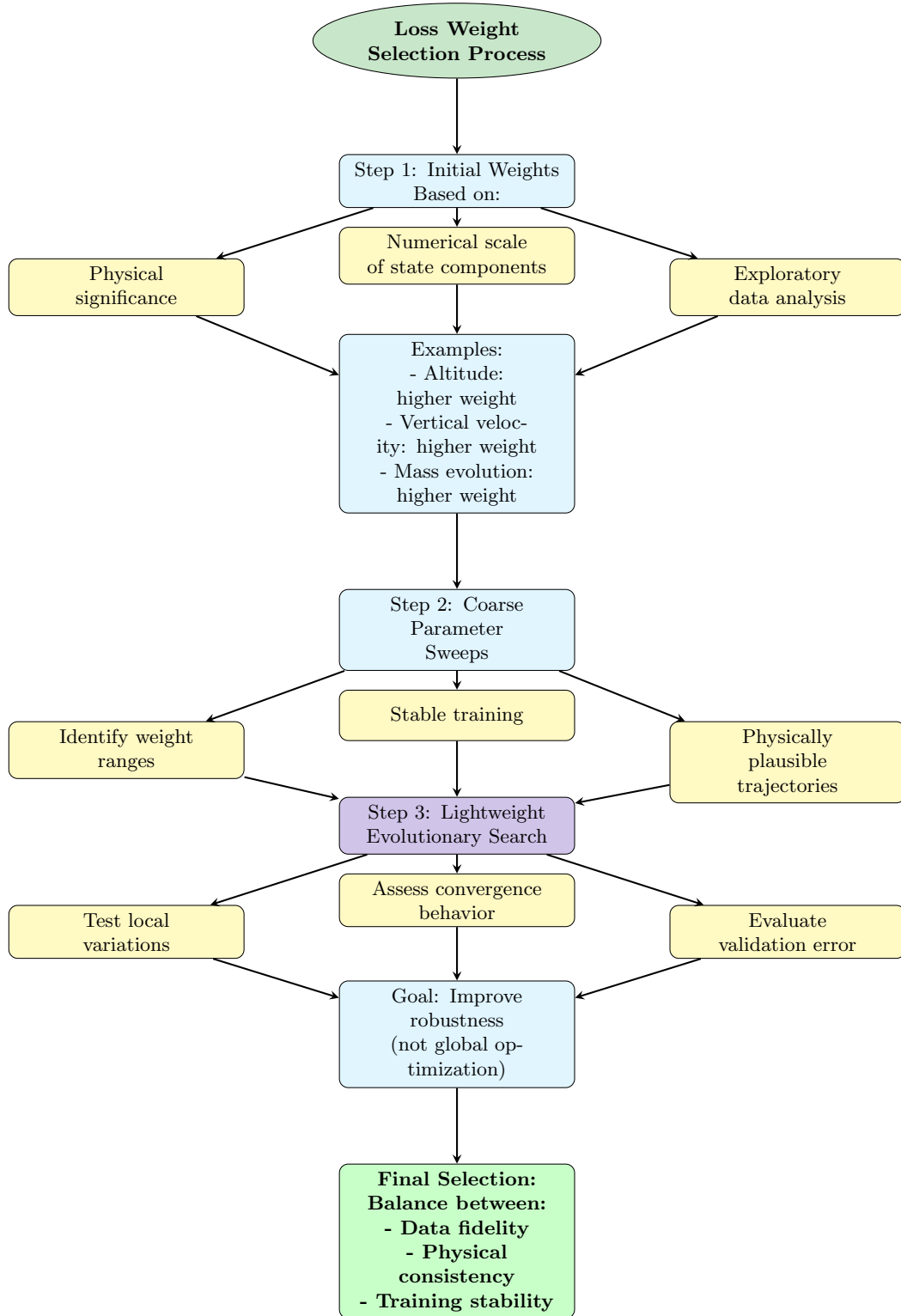


Figure 2.4: Loss weight selection process. The selection of loss weights followed a structured procedure summarized, combining physical intuition, sensitivity analysis, and local evolutionary refinement to ensure stable and interpretable training.

training. Optimal control trajectories were generated using direct collocation with Hermite–Simpson discretization and solved using IPOPT. These trajectories were used to construct training datasets with 120 training cases, 20 validation cases, and 20 test cases, each discretized on a uniform time grid with 1501 points over 30 seconds. A physics-informed neural network architecture (Direction AN) was designed with a shared stem, three specialized branches for translation, rotation, and mass, and a physics residual layer. The network was trained using a phased schedule with Adam optimization, incorporating data fidelity, physics-based regularization, structural constraints, and smoothing regularization terms in the loss function.

CHAPTER III

RESULTS

3.1 Evaluation Setup

This chapter evaluates the performance of the proposed PINN-inspired physics-regularized neural surrogate using a held-out test set that was not used during training. To evaluate how well trajectories are reconstructed, examine numerical error patterns alongside adherence to selected physical constraints.

The evaluation set includes several numerically generated ascent trajectories for rockets employing the same high-fidelity numerical integration framework described in Chapter II. Every path over a set period of thirty seconds, including movement-related conditions, is part of the evaluation set. States of rotation are described through quaternions, while changes in vehicle mass unfold over time. All trajectories are sampled at a uniform temporal resolution of 50 Hz, consistent with the training setup.

Prediction accuracy is evaluated by comparing the PINN outputs against the reference solver outputs at corresponding time instances. Absolute error is computed pointwise in time for representative trajectories, while root-mean-square error is used to quantify performance across the entire test set. All reported RMSE values are computed per trajectory and then aggregated across the test dataset.

No control inputs or reference trajectories are provided to the network during inference. The model predicts the full state evolution solely from time and mission context parameters, without access to control inputs or reference trajectories during inference.

3.2 Trajectory Reconstruction Results

This section presents a qualitative comparison between the PINN-predicted trajectory and the reference trajectory for a single representative test case. The purpose is to visually assess how well the model reconstructs the time evolution of different state components.

3.2.1 Vertical Motion

The vertical motion results demonstrate the model’s ability to reconstruct position and velocity trajectories. As shown in Figure 3.1, the predicted vertical position closely follows the reference solution over the full ascent phase, with a visible but small deviation near the end of the time horizon. Figure 3.2 presents the corresponding vertical velocity profile, where the predicted velocity captures the overall trend and curvature of the reference trajectory, with noticeable divergence during the later phase of motion.

3.2.2 Mass evolution

The mass evolution results highlight the model’s ability to approximate the overall trend of propellant consumption during the burn phase and the constant-mass regime after cutoff, as shown in Figure 3.3. Small oscillations around the terminal mass, visible in the figure, indicate limitations of soft constraint enforcement rather than numerical instability.

3.2.3 Horizontal motion

The horizontal motion results illustrate small deviations from the zero-motion reference trajectory, as shown in Figure 3.4. These deviations reflect imperfect symmetry preservation and provide insight into the limitations of penalty-based constraints in unconstrained dimensions.

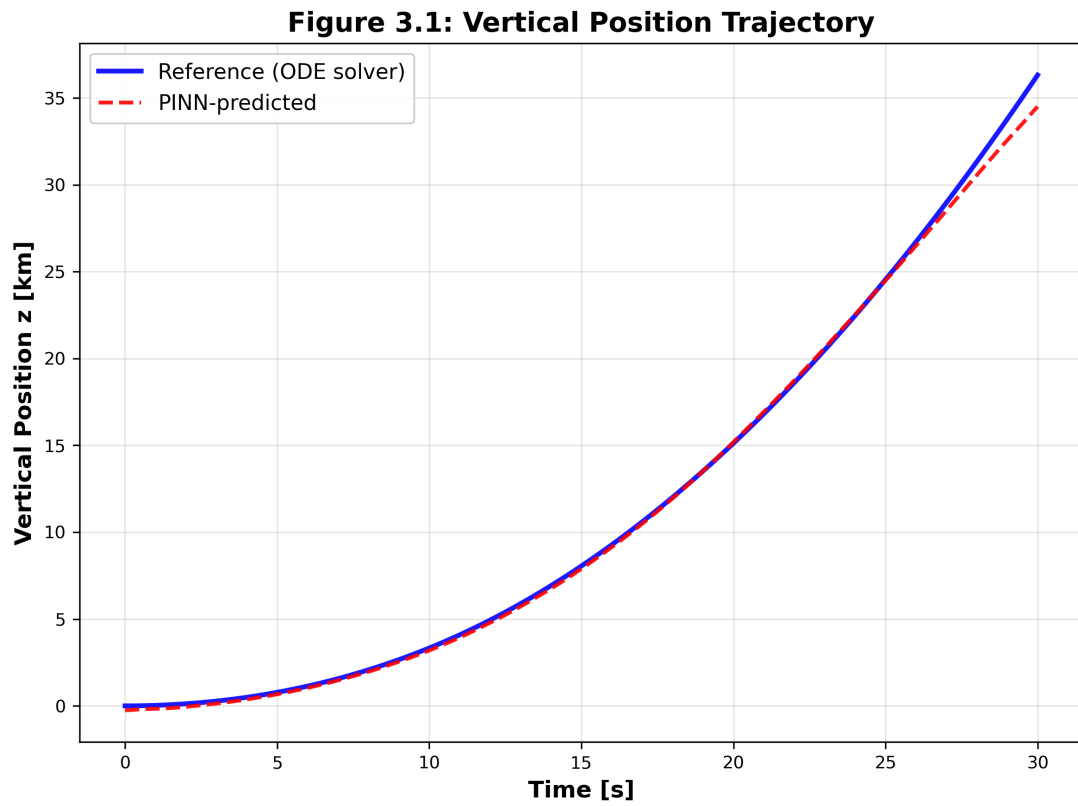


Figure 3.1: Vertical position: Shows the vertical position as a function of time. The PINN prediction closely follows the reference solution over the full ascent phase, with a visible but small deviation near the end of the time horizon.

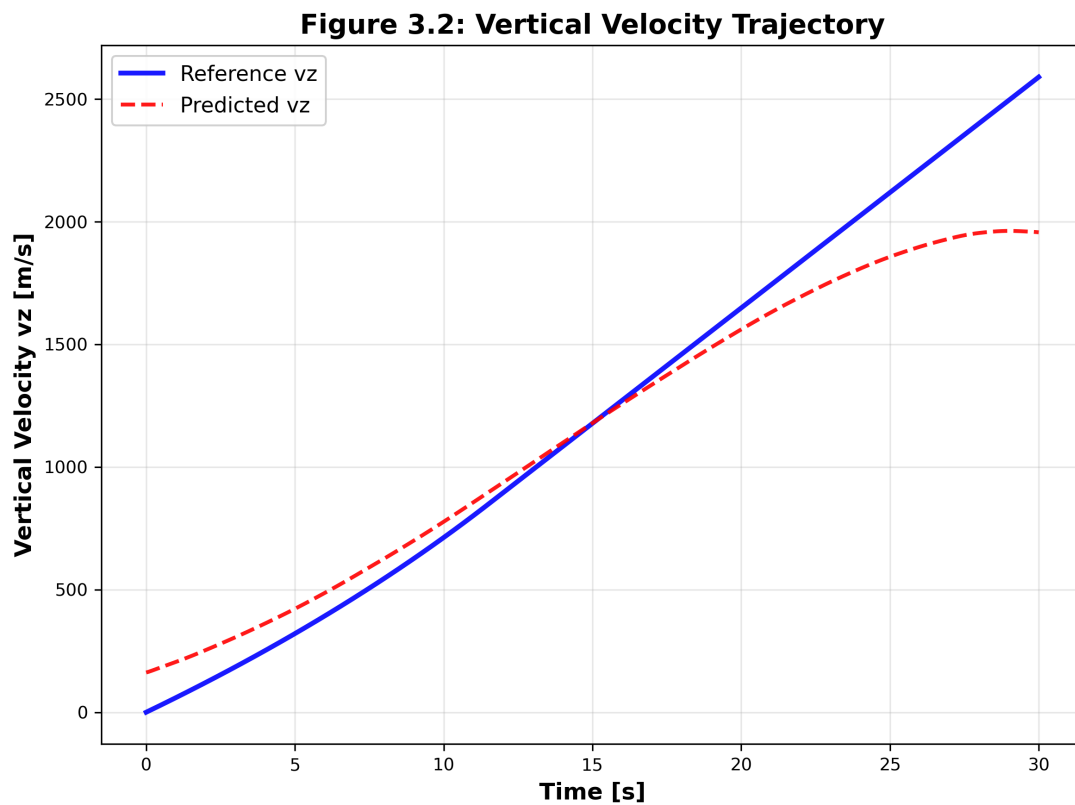


Figure 3.2: Vertical velocity: Presents the corresponding vertical velocity profile. The predicted velocity captures the overall trend and curvature of the reference trajectory, with noticeable divergence during the later phase of motion.

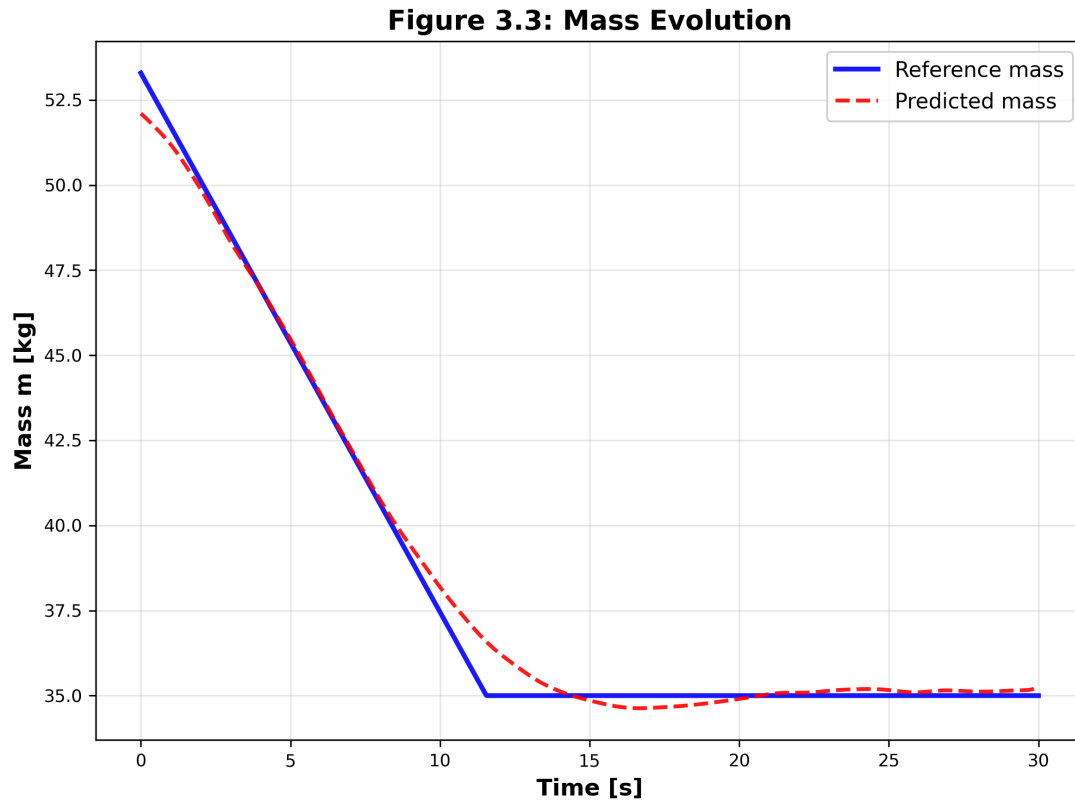


Figure 3.3: Mass evolution: Compares the predicted mass evolution against the reference solution. During the propulsive phase, although the forecasted values drop steadily like the standard curve, minor variations appear once fuel runs out. While the actual mass stabilizes completely, the model's output shows slight deviations near the endpoint. These differences persist without growing larger over time.

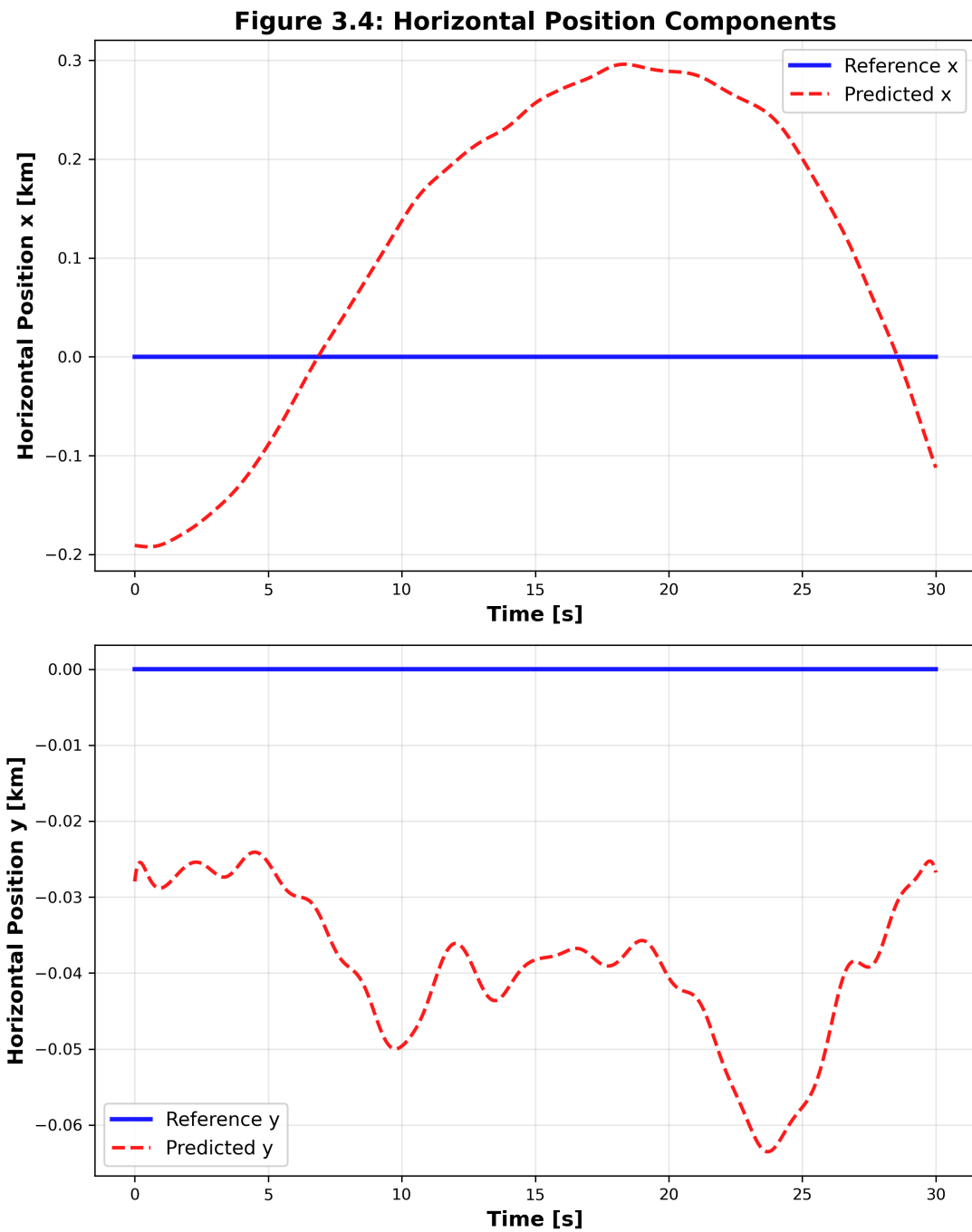


Figure 3.4: Horizontal position: Shows how position changes sideways. In both left-right directions, the target path stays flat at zero. Instead, the forecasted route shifts slightly, showing movement off course. This shift appears because weak constraints affect symmetric systems more strongly. Sensitivity grows when balance rules lack tight control.

3.2.4 Attitude representation

The attitude results demonstrate stable quaternion behavior throughout the trajectory, as shown in Figure 3.5. Explicit normalization effectively prevents drift in the orientation representation, ensuring physically meaningful rotational states, which is confirmed by the extremely small deviation from unity observed in the figure.

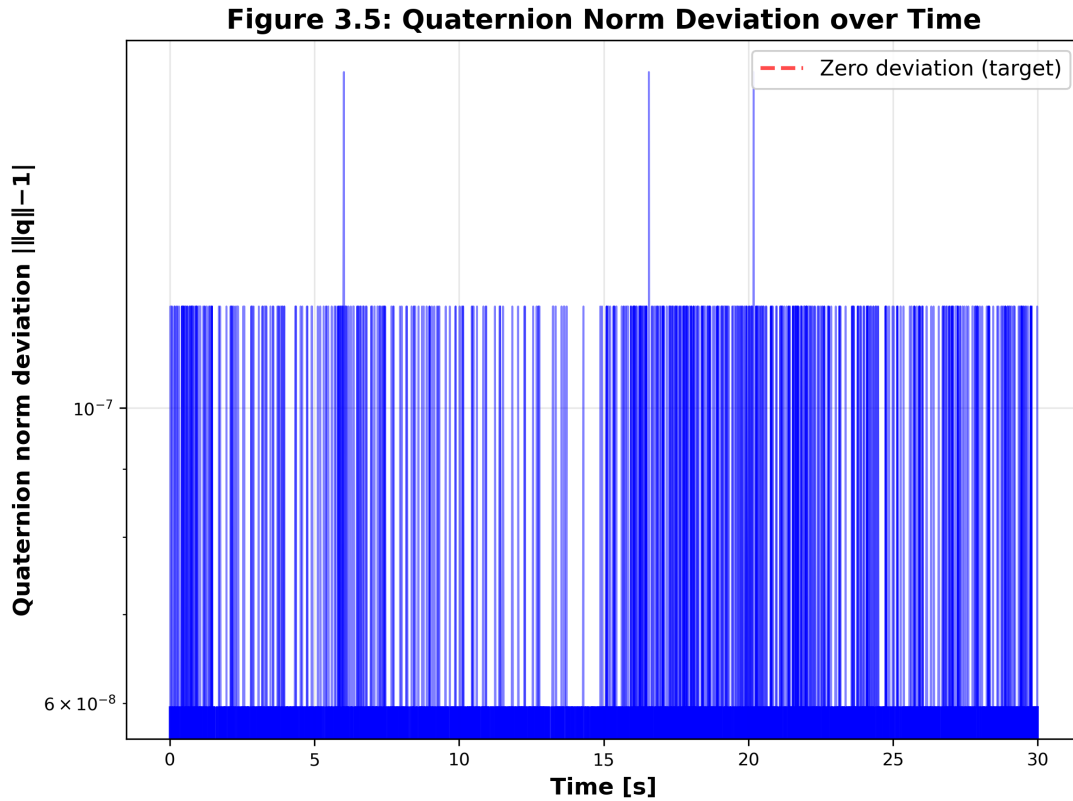


Figure 3.5: Quaternion norm: Show how closely the predicted quaternion norm stays to one over time. Despite minor fluctuations, the value hardly drifts away from one, which suggests normalization holds well across the entire trajectory.

3.3 Quantitative Error Metrics

This section evaluates prediction accuracy across the entire test set using quantitative error measures.

State Variable	Mean RMSE	Standard Deviation	Units
x	0.019133	0.000854	nondim
y	0.002535	0.001278	nondim
z	0.065454	0.015009	nondim
vx	0.275908	0.017068	nondim
vy	0.002568	0.001137	nondim
vz	0.575095	0.080124	nondim
q0	0.072533	0.029386	nondim
q1	0.000960	0.000707	nondim
q2	0.335586	0.071237	nondim
q3	0.000720	0.000347	nondim
wx	0.000801	0.000251	nondim
wy	0.008613	0.001579	nondim
wz	0.000704	0.000175	nondim
m	0.012871	0.007033	nondim

Table 3.1: The root-mean-square error for each state variable, averaged across 20 test trajectories. Where position, velocity, rotation, and mass errors are reported separately to reflect their different physical scales.

3.3.1 Per-state RMSE statistics

3.3.2 Aggregate performance measures

Metric	Observed Range	Test Set Size
Vertical position accuracy (RMSE)	0.067152 ± 0.015009	20
Velocity accuracy (RMSE)	0.580650 ± 0.080124	20
Mass behavior (RMSE)	0.014667 ± 0.007033	20
Quaternion norm range	[1.000000, 1.000000]	20
Residual magnitude range	[0.166489, 9.513170]	20

Table 3.2: Summary of overall error statistics, including mean and dispersion measures across 20 test trajectories, created an overview of prediction accuracy beyond individual state variables.

3.3.3 RMSE distribution

3.4 Constraint Satisfaction Results

This section examines whether the model’s predictions satisfy key physical constraints imposed during training. By plotting the time evolution of the physics resid-

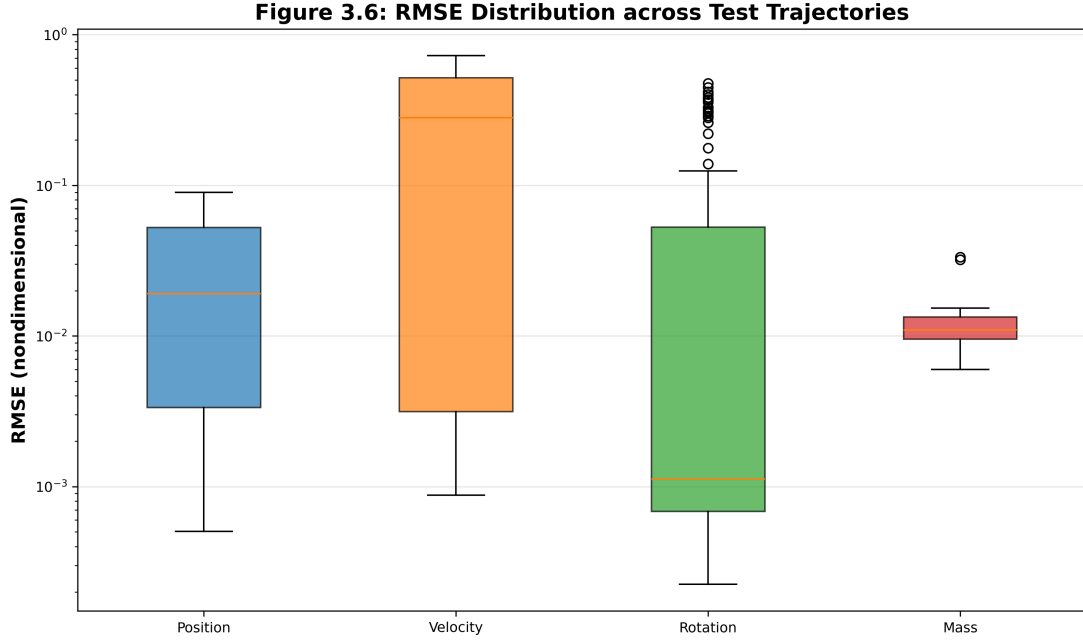


Figure 3.6: The distribution of RMSE values across 20 test trajectories for different state groups, created a logarithmic scale to highlight variability and outliers in prediction performance.

ual norm, we can see how well the model’s predictions satisfy the governing equations of motion.

3.4.1 Physics residual norm

Figure 3.7 shows the time evolution of the physics residual norm for the representative trajectory. The residual magnitude varies over time and exhibits a notable decrease toward the end of the trajectory, as shown in the figure. The reduction in residual magnitude should not be interpreted as uniform improvement in physical accuracy, but as a consequence of reduced actuation and simplified dynamics after thrust termination.

3.4.2 Mass monotonicity constraint

Figure 3.8 evaluates compliance with the mass monotonicity constraint by plotting the time derivative of mass. The reference trajectory shows no violations, while the predicted trajectory exhibits multiple instances where the monotonicity condition is

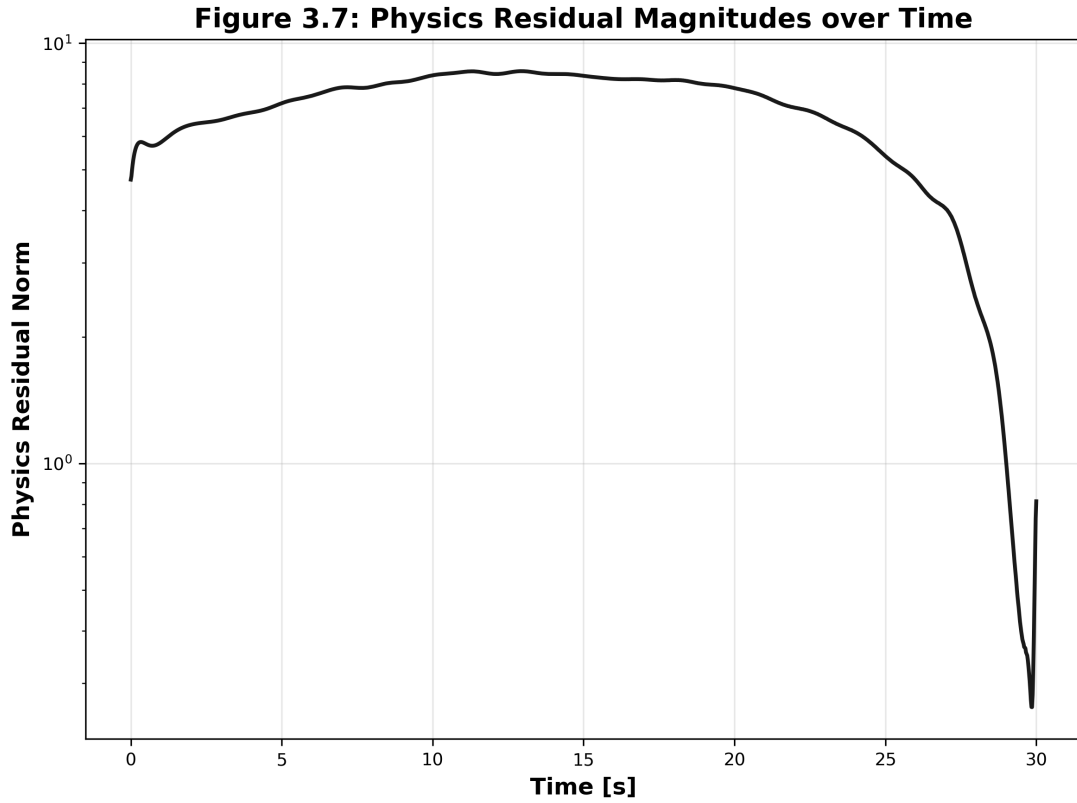


Figure 3.7: Time evolution of the physics residual norm for the representative trajectory. The residual magnitude varies over time and exhibits a notable decrease toward the end of the trajectory. The reduction in residual magnitude should not be interpreted as uniform improvement in physical accuracy, but as a consequence of reduced actuation and simplified dynamics after thrust termination.

violated, as shown in the figure.

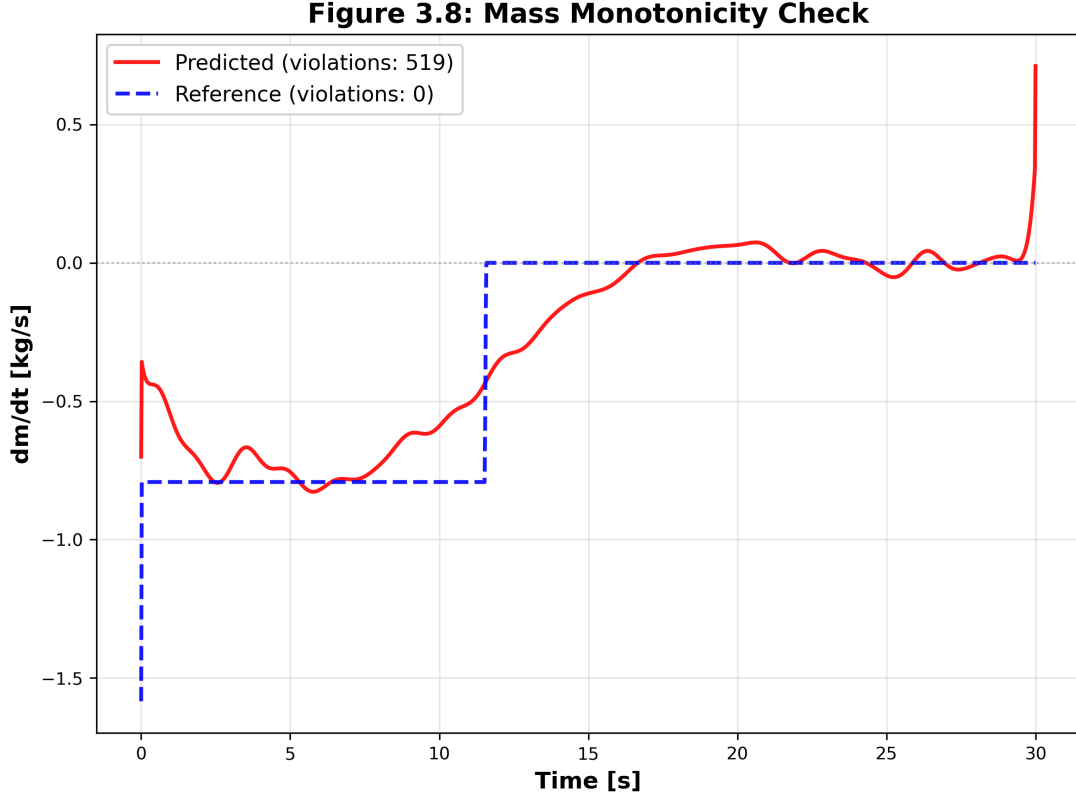


Figure 3.8: evaluates compliance with the mass monotonicity constraint by plotting the time derivative of mass. The reference trajectory shows no violations, while the predicted trajectory exhibits multiple instances where the monotonicity condition is violated.

3.5 Summary of Observed Results

In this chapter, we demonstrate that the proposed neural surrogate is capable of reconstructing the overall structure of rocket trajectories, as shown by close visual agreement in vertical position and velocity in Figures 3.1 and 3.2. Quantitative evaluation across the test set, as summarized in Table 3.1 and Figure 3.6, indicates that prediction errors vary by state type, with mass and rotational states exhibiting distinct error distributions. Quaternion normalization was maintained throughout the trajectory via constant satisfaction of the quaternion norm as shown in Figure 3.5, while mass monotonicity violations occur in the predicted trajectories, as illustrated

in Figure 3.8. The physics residuals remain bounded and decrease toward the end of the simulated time horizon, as shown in Figure 3.7.

All observations reported in this chapter are based solely on empirical evaluation results without interpretation of underlying causes.

CHAPTER IV

DISCUSSION AND IMPLICATIONS

4.1 Overall Performance of the Neural Surrogate

The results indicate that the proposed neural surrogate is able to recover the overall structure of rocket trajectories when tested on previously unseen cases. In particular, the model reproduces the main trends in vertical position, velocity, and mass evolution, even though explicit control inputs are not provided during the inference stage. This outcome suggests that physics-based regularization terms, derived from the governing equations, introduce a useful inductive bias that supports trajectory reconstruction in complex, high-dimensional dynamical systems. It should be noted, however, that the proposed approach focuses on approximating optimal ascent trajectories, rather than learning a general closed-form state transition model. This limitation arises because the proposed approach approximates optimal ascent trajectories directly, rather than learning a general closed-form state transition model.

The strong agreement observed in the vertical position throughout most of the ascent phase further indicates that the neural network effectively captures the accumulated effects of thrust and acceleration over time. The behavior of a PINN-inspired model reflects typical observations in the literature concerning physics-informed training, in that reconstructed state variables follow smooth integral-structured dynamics and are usually reconstructed with greater accuracy than predicted values requiring a direct time derivative.

However, substantial errors developing at the end of the trajectory indicate that error accumulation still presents a challenge for long-term predictions. This observation has previously been demonstrated through research conducted using PINNs

and their variants applied to nonlinear ordinary differential equations, in which it was found that the magnitude of the prediction error increased as the distance in time from the initial condition increased. Consequently, although the general shape of the trajectory was maintained, local inaccuracies became more pronounced at later stages in the motion.

4.2 Interpretation of Velocity and Higher-Order Dynamics

The vertical velocity results exhibit noticeably larger discrepancies than those observed for position, particularly during the later stages of the flight. This behavior is not unexpected, since velocity is obtained directly from first-order time derivatives of the network output. Since we take model-inspired by PINN, such derivative-based quantities are generally more sensitive to approximation errors and to stiffness effects that arise during training, which makes accurate reconstruction more difficult.

The growing divergence in predicted velocity near the terminal phase further suggests that the network tends to emphasize overall trajectory coherence rather than strict local accuracy of derivatives. In other words, the learning process appears to favor consistency at the global level, even if this comes at the expense of precise matching of velocity profiles at specific time points. This type of trade-off has been widely reported in studies of PINNs or its variants, where the minimization of combined data and physics losses does not necessarily lead to uniform accuracy across all state variables and their derivatives.

From a physical standpoint, velocity evolution is governed by forces that can change rapidly over short time intervals, such as thrust termination and variations in mass. Accurately resolving these effects typically requires either finer temporal sampling or a stronger and more targeted enforcement of the governing equations during such transition phases. While the current neural surrogate demonstrates acceptable overall behavior, these results indicate that velocity prediction remains a

limiting factor when high-fidelity trajectory reconstruction is required.

4.3 Mass Evolution and Constraints Enforcement

The mass evolution results show that the network learns the general trend of propellant consumption during the burn phase and the constant-mass regime after cutoff. However, small oscillations around the terminal mass and multiple monotonicity violations are observed in the predicted trajectory.

These violations indicate that while the mass conservation law is included as a physics residual, it is not enforced as a hard constraint. As a result, the neural network allows small physically inconsistent fluctuations if they reduce the overall loss. This behavior highlights an important limitation of soft constraint enforcement in PINNs or in this case, physics-regularized neural surrogates. This behavior highlights a fundamental limitation of penalty-based physics enforcement and underscores the distinction between physics-informed learning and physics-guaranteed modeling.

In practical aerospace applications, mass monotonicity is a non-negotiable physical constraint. The current results suggest that additional structural constraints or post-processing steps are necessary if the model is to be deployed in safety-critical settings. Nonetheless, the ability of the neural network to approximate the mass profile without explicit integration demonstrates the flexibility of the physics-informed approach.

4.4 Horizontal Drift and Symmetry Breaking

The horizontal position results show small yet persistent deviations from the reference trajectory, which is identically zero throughout the motion. This behavior demonstrates that the neural network does not necessarily maintain symmetry along dimensions of the problem not specifically restricted, even though the physical solution corresponding to zero displacement is expected to preserve this symmetry.

In neural network-based models of differential equations, drift is a common occurrence, particularly with respect to governing equations with trivial and/or stationary solutions. Without implementing specific penalties or restrictions to ensure symmetry within an optimization process, the resulting convergence may satisfy the overall loss function while remaining inconsistent with the defined symmetry of the physical problem at hand. In this manner, the trained model meets the training goal, but does not strictly adhere to the physics-associated symmetry.

This finding implies that it may be beneficial to include regularization terms or coordinate-specific constraints when modeling physical problems that require zero-motion states due to physics. Alternatively, if identified, the model formulation could be simplified by eliminating extraneous variables (i.e., degrees of freedom) resulting in less spurious predictions and improved robustness. This synergistic behavior would be enhanced by the lack of explicit control inputs applied to the residual form of the problem, as it means that enforcement of lateral force balance is not applied under current conditions.

4.5 Quaternion Normalization and Attitude Representation

The quaternion norm deviation results indicate that throughout the entire predicted trajectory the predicted orientation remains close to a unit norm, which suggests that the normalization techniques used during training were properly implemented and allowed for the effective maintenance of accurate and consistent attitude estimates in the prediction process.

This is an important feature of rotational dynamics, as even small departures from a unit norm can accumulate and produce attitudes that do not correspond to the physical motion of a body. Thus, the results obtained indicate that the formulation developed limits the occurrence of such deviations, while at the same time being stable enough to propagate without introducing numerical instability.

These results also support the findings in the literature regarding learning-based

attitude models, where explicit normalization schemes or penalties are needed to maintain physically plausible orientations. The present results provide further evidence for the effectiveness of the approach chosen in this literature.

4.6 Physics Residual Behavior and Model Consistency

The examination of the residuals from the physics model provides a better understanding of how accurately the predicted solution matches with what would normally be predicted by the governing equations of motion in the standard way that a flight would behave. In general, the residual magnitudes stay within reasonable bounds over the duration of the simulation, and as time moves forward in the simulation, the availability of the force values decreases to the minimum values at the final time step. This pattern suggests that as time passes, the model is becoming progressively more consistent with the governing equations of motion.

However, the residuals do not have the same profile throughout the trajectory. The variability in the residuals from the beginning of flight through the end of flight demonstrates that there is a changing level of satisfaction of the constraints imposed by the equations of motion between different flight phases. This level of variability in the satisfaction of the constraints imposed by the equations of motion is further accentuated by the absence of explicit control inputs in the formulation of residuals. Therefore, since the lateral force balance from the control inputs is not explicitly imposed, it becomes possible for certain components of the residual to dominate during different phases of motion.

In the literature regarding PINNs and other approaches that utilize learning for system identification in nonlinear systems with time-varying dynamics, there has been evidence of this behavior. Each operating condition will have a different residual term that becomes prominent for that condition due to a shift in the balance of dominant physical phenomena. This illustrates the relevance of techniques such as adaptive loss weighting, or curriculum training, which are often designed to help al-

leviate the problem of consistently enforcing constraints when using these methods for modeling systems across multiple operational ranges.

4.7 Practical Implications and Applicability

On the basis of how the results obtained show, physics-regularized neural surrogate will provide a promising avenue for surrogate modelling for rocket trajectory reconstruction in an analysis context. The ability to provide complete state trajectory information without requiring discrete numerical integration points will benefit many applications of the approach, including rapid simulation tasks, early-stage design space exploration, and aircraft embedded onboard prediction systems where fast evaluations are required. Thus, the approach has the potential to be used in a wide range of applications, including but not limited to, trajectory optimization, trajectory tracking, and trajectory prediction.

However, the existence of constraint violations and significant inaccuracies in the derivative estimates indicate that the current formulation is primarily intended for analysis and approximation purposes, and not for direct use in closed-loop control systems. Thus, in real-world applications (including industrial deployment and mission-critical operations), it would be prudent to implement additional layers of verification, validation, and safety mechanisms prior to practical implementation (e.g. field testing or laboratory testing).

Nonetheless, the proposed approach is compelling in scenarios whereby computational efficiency and differentiability are of utmost importance. Optimization problems, inverse modelling, and sensitivity analyses are cases where the desired properties of the proposed approach are advantageous, albeit not all stages need to conform to strict physical fidelity. Thus, the approach is not limited to the analysis and approximation purposes, and can be used in a wide range of applications, including but not limited to, trajectory optimization, trajectory tracking, and trajectory prediction.

4.8 Limitations of the Present Study

Several limitations of the present study should be clearly recognized. First, the model is assessed using simulated data that are generated from the same physical assumptions employed during training. As a result, the ability of the approach to generalize to real flight data, where disturbances, uncertainties, and modeling errors are inevitably present, has not yet been examined.

Second, all physical constraints are enforced through soft penalty terms in the loss function. While this strategy offers flexibility during optimization, it also permits solutions that are not fully physically consistent when the training process prioritizes overall loss reduction. This effect can be observed most clearly in behaviors such as violations of mass monotonicity and the appearance of horizontal drift, as discussed in the previous sections.

Third, using a fixed timeframe and equal time intervals may also limit the ability to model different kinds of flights or abrupt changes in dynamics. These design choices make it easier to train and evaluate a model, but may limit the model's ability to adapt to more complicated situations.

The above limitations do not invalidate the outcomes obtained from this research. However, it does indicate that there are many parts of this framework that need to be improved upon in future work.

4.9 Recommendations for Future Improvements

Hybrid constraint enforcements can be created using physics-based loss terms and having hard constraints or projection-based correction methods. Specifically, properties such as mass monotonicity and symmetry being enforced directly in the model would likely increase the model's quality of physical consistency. This approach is preferable over relying on penalties throughout training.

Furthermore, adaptive loss weighting schemes and phase-aware training strate-

gies have the potential to increase model performance in critical dynamic transitions, such as engine cutoff, where the governing dynamics are suddenly altered. To allow for greater flexibility and practicality of this framework, the incorporation of variable time horizons and stochastic disturbances into this framework must also be included.

Finally, the use of real-world flight data or higher-fidelity aerodynamic ratios will also enable a much more rigorous test of robustness and generalization of the hybrid constraint enforcement as described above. Including these data will permit evaluation of the proposed hybrid constraint enforcement in more realistic situations, therefore providing developed evidence for its applicability outside of controlled testing settings.

CHAPTER V

CONCLUSION AND RECOMMENDATIONS

5.1 Conclusions

This thesis examined whether a structured, physics-regularized neural surrogate is capable of serving as an effective surrogate model for reconstructing rocket ascent trajectories, and whether such a model can be integrated into a broader trajectory optimization framework if extended to explicitly condition on control inputs and enforce full dynamics. The study was motivated by the computational cost and limited flexibility of classical optimal control solvers when repeated trajectory evaluations or gradient information are required.

The results support the conclusion that physics-regularized neural surrogates constitute a promising surrogate modeling approach for rocket ascent trajectory reconstruction when the governing equations are well understood and training data are available. By incorporating physics-based regularization terms derived from the equations of motion into the learning objective, the proposed model is able to reconstruct full-state trajectories that are globally consistent with the dominant trends of the reference dynamics, without relying on explicit numerical integration during inference. This suggests that physics guidance can significantly improve the reliability of neural surrogates compared to purely data-driven models. All conclusions are restricted to simulated trajectories generated from the same modeling assumptions used during training.

In line with prior work on physics-informed learning, the study shows that state variables governed primarily by integral effects are captured more robustly than derivative-dominated quantities. This observation is consistent with findings re-

ported by Raissi et al. and Karniadakis et al., where PINNs were shown to approximate smooth solution manifolds effectively while remaining sensitive to higher-order dynamics. The present work extends these insights to a six-degree-of-freedom rocket ascent problem with coupled translational, rotational, and mass dynamics.

At the same time, the thesis shows that relying solely on soft enforcement of physical constraints is not sufficient to ensure strict adherence to non-negotiable physical laws, such as mass monotonicity and symmetry preservation. Although the model is able to learn the overall structure of propellant consumption and the main characteristics of orientation dynamics, local violations can still occur when constraint satisfaction competes with data fidelity during training. This observation is consistent with recent theoretical analyses of physics-regularized neural surrogates optimization behavior, which highlight the inherent trade-offs between different loss components and the lack of hard guarantees when constraints are imposed only through penalty-based formulations.

The analysis shows that physics-constrained neural network substitutes should be utilized preferably for analysis/optimization rather than used directly as replacements for high-fidelity simulation in high-risk (safety critical) applications. The primary advantage of this new class of surrogate models is their ability to provide computationally efficient approximations that preserve many of the characteristics of actual complex dynamical systems; however, local-level verification of exact compliance with physics principles cannot always be guaranteed (e.g. linear simulation errors can accumulate with increased complexity.)

5.2 Implications for Trajectory Optimization and Scientific Machine Learning

The proposed surrogate approach highlights a potential path to decreasing reliance on repeated numeric integrations within the gradient-based optimization framework from a trajectory optimization standpoint. The model is entirely differentiable,

hence allowing for direct computation of the initial condition sensitivities via formal derivative methods. Moreover, once control inputs are integrated into the model, we may formally compute the control parameter sensitivities. Generally, deriving these types of sensitivity analyses is challenging when using standard numerical solvers because sensitivity analyses require many evaluations of the problem.

In a broader context, within scientific machine learning, this research supports the viewpoint that the structural architecture of a neural network and a neural network’s loss function formulation may be as significant as the overall neural network capacity or ability. The examples presented in this research suggest that performance is very dependent on the manner in which the training objectives of the model and the predictions to be made by the model are aligned. The findings presented in this study provide practical support to ongoing efforts to develop PINNs and physics-constrained neural network models regarding issues related to robustness, constraint enforcement, and long-term prediction capabilities.

5.3 Recommendations for Future Work

Many recommendations for future work have been derived from the results of this research.

To begin with, when creating future models, it is recommended that future work look to include hard or semi-hard enforcement mechanisms alongside purely penalty-based loss functions placing constraints on critical physical laws such as mass monotonicity and symmetry conditions. Several approaches, including projection or constrained output parameterization methods and hybrid solver networks are recommended to employ the hard or semi-hard enforcement mechanisms. These types of approaches may provide a much more reliable and accurate expectation for the enforcement of the critical physical constraints than soft enforcement methods alone, particularly in those situations where it is unacceptable to accept the possibility of a violation of the critical physical constraints.

The second recommendation is to utilize either phase-aware or adaptive training strategies. This would allow rapid dynamic change periods (such as engine shutoff and thrust transition periods) to be handled more effectively. Dynamic adjustment to the loss weight during training or introducing a curriculum-based method of training could improve the accuracy of the derivative estimates while still maintaining consistency in trajectory profile over the entire time horizon.

Third, extending the framework to accommodate varying time horizons as well as non-nominal flight conditions (such as wind disturbance and parameter uncertainty) would allow for much greater relevance of the framework to real-world applications. In addition to increasing the practical relevance of the framework, it would also allow for much greater evaluation of how well the developed model generalizes to conditions that were not represented during training.

Finally, in the future work, it is highly recommended to combine experimental flight data and/or a more accurate simulation environment to evaluate the feasibility and effectiveness of this approach. To this end, if possible, combining physics-based neural networks with data assimilation techniques to construct hybrid models may provide an excellent opportunity to close the gap between simulation only models and models used for actual flight operations.

5.4 Final Remarks

This thesis demonstrates that by sufficiently designing and carefully evaluating Physics-Regularized Neural Surrogates, it can be meaningfully used as a model to reconstruct trajectory for rocket launch and ascent. While the PINN-inspired paradigms will not replace traditional simulation tools, they provide an alternative approach that can assist with quick model evaluations of desirable properties, allow for differentiation in some sense, and possess sufficient physical properties during learning of the trajectory.

The results from this study provide evidence supporting both strengths and lim-

itations of Physics-Regularized Neural Surrogates as well as an avenue to continue to investigate the ability to use Machine Learning in a deeper way in trajectory analysis/optimization, and in addition to develop a broader Machine Learning Scientific Workflow.